NetflixScope -- Exploratory Data Analysis of Netflix Movies & TV Shows

Subject: Advance Programming Lab-I (Python)
Subject Code: CSE(AI/CPS/CS/DS)-3006

Name	Scholar No.
Anupam Bichpuria	25U022014
Ishu Nayak	25U022004

---

1. Introduction

Netflix is one of the largest media and video streaming platforms in the world. As of mid-2021, the platform hosted over 8,000 movies and TV shows. Understanding the composition of this content library -- the mix of movies versus TV shows, the dominant genres, the geographic spread of productions, and temporal trends -- is valuable for data-driven decision-making.

Objectives

1. Understand the structure, quality, and distribution of the Netflix dataset.
2. Clean and preprocess the data for reliable analysis.
3. Perform univariate, bivariate, and multivariate analyses.
4. Visualize key patterns and trends using charts.
5. Summarize findings, acknowledge limitations, and draw conclusions.

Dataset Description

The dataset netflix_titles.csv contains information about Movies and TV Shows available on Netflix. Each row represents a single title with attributes such as type, title, director, cast, country, date added, release year, rating, duration, genres, and a brief description.

2. Setup -- Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 100

print("Libraries loaded successfully.")
```

```
Libraries loaded successfully.
```

3. Dataset Loading and Inspection

```
df = pd.read_csv('netflix_titles.csv')
print(f"Dataset shape: {df.shape[0]} rows x {df.shape[1]} columns")
df.head()
```

```
Dataset shape: 8807 rows x 12 columns


  show_id     type                  title         director  \
0      s1    Movie   Dick Johnson Is Dead  Kirsten Johnson
1      s2  TV Show          Blood & Water              NaN
2      s3  TV Show              Ganglands  Julien Leclercq
3      s4  TV Show  Jailbirds New Orleans              NaN
4      s5  TV Show           Kota Factory              NaN

                                          cast        country  \
0                                          NaN  United States
1  Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...   South Africa
2  Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...            NaN
3                                          NaN            NaN
4  Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...          India

           date_added  release_year rating   duration  \
0  September 25, 2021          2020  PG-13     90 min
1  September 24, 2021          2021  TV-MA  2 Seasons
2  September 24, 2021          2021  TV-MA   1 Season
3  September 24, 2021          2021  TV-MA   1 Season
4  September 24, 2021          2021  TV-MA  2 Seasons

                                     listed_in  \
0                                Documentaries
1  International TV Shows, TV Dramas, TV Mysteries
2  Crime TV Shows, International TV Shows, TV Act...
3                            Docuseries, Reality TV
4  International TV Shows, Romantic TV Shows, TV ...

                                    description
0  As her father nears the end of his life, filmm...
1  After crossing paths at a party, a Cape Town t...
2  To protect his family from a powerful drug lor...
3  Feuds, flirtations and toilet talk go down amo...
4  In a city of coaching centers known to train I...
```

```
df.tail()
```

```
     show_id     type        title         director  \
8802   s8803    Movie       Zodiac    David Fincher
8803   s8804  TV Show  Zombie Dumb              NaN
8804   s8805    Movie   Zombieland  Ruben Fleischer
8805   s8806    Movie         Zoom     Peter Hewitt
8806   s8807    Movie       Zubaan      Mozez Singh

                                           cast        country  \
8802  Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...  United States
8803                                        NaN            NaN
8804  Jesse Eisenberg, Woody Harrelson, Emma Stone, ...  United States
8805  Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...  United States
8806  Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...          India

             date_added  release_year rating   duration  \
8802  November 20, 2019          2007      R    158 min
8803       July 1, 2019          2018  TV-Y7  2 Seasons
8804   November 1, 2019          2009      R     88 min
8805   January 11, 2020          2006     PG     88 min
8806      March 2, 2019          2015  TV-14    111 min

                                     listed_in  \
```

```
8802                  Cult Movies, Dramas, Thrillers
8803          Kids' TV, Korean TV Shows, TV Comedies
8804                          Comedies, Horror Movies
8805              Children & Family Movies, Comedies
8806  Dramas, International Movies, Music & Musicals

                                           description
8802  A political cartoonist, a crime reporter and a...
8803  While living alone in a spooky town, a young g...
8804  Looking to survive in a world taken over by zo...
8805  Dragged from civilian life, a former superhero...
8806  A scrappy but poor boy worms his way into a ty...
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   str   
 1   type          8807 non-null   str   
 2   title         8807 non-null   str   
 3   director      6173 non-null   str   
 4   cast          7982 non-null   str   
 5   country       7976 non-null   str   
 6   date_added    8797 non-null   str   
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   str   
 9   duration      8804 non-null   str   
 10  listed_in     8807 non-null   str   
 11  description   8807 non-null   str   
dtypes: int64(1), str(11)
memory usage: 3.9 MB
```

4. Data Types and Descriptive Statistics

```
print("Data types:\n")
print(df.dtypes)
```

```
Data types:

show_id           str
type              str
title             str
director          str
cast              str
country           str
date_added        str
release_year    int64
rating            str
duration          str
listed_in         str
description       str
dtype: object
```

```
df.describe()
```

```
       release_year
count   8807.000000
```

```
mean    2014.180198
std        8.819312
min     1925.000000
25%     2013.000000
50%     2017.000000
75%     2019.000000
max     2021.000000
```

```
df.describe(include='object')
```

```
       show_id   type                  title       director  \
count     8807   8807                   8807           6173
unique    8807      2                   8807           4528
top         s1  Movie  Dick Johnson Is Dead  Rajiv Chilaka
freq         1   6131                      1             19

                     cast        country       date_added rating  duration  \
count                7982           7976             8797   8803      8804
unique               7692            748             1767     17       220
top     David Attenborough  United States  January 1, 2020  TV-MA  1 Season
freq                   19           2818              109   3207      1793

                          listed_in  \
count                          8807
unique                          514
top     Dramas, International Movies
freq                            362

                                                 description
count                                                   8807
unique                                                  8775
top     Paranormal activity at a lush, abandoned prope...
freq                                                       4
```

5. Missing-Value Analysis and Handling

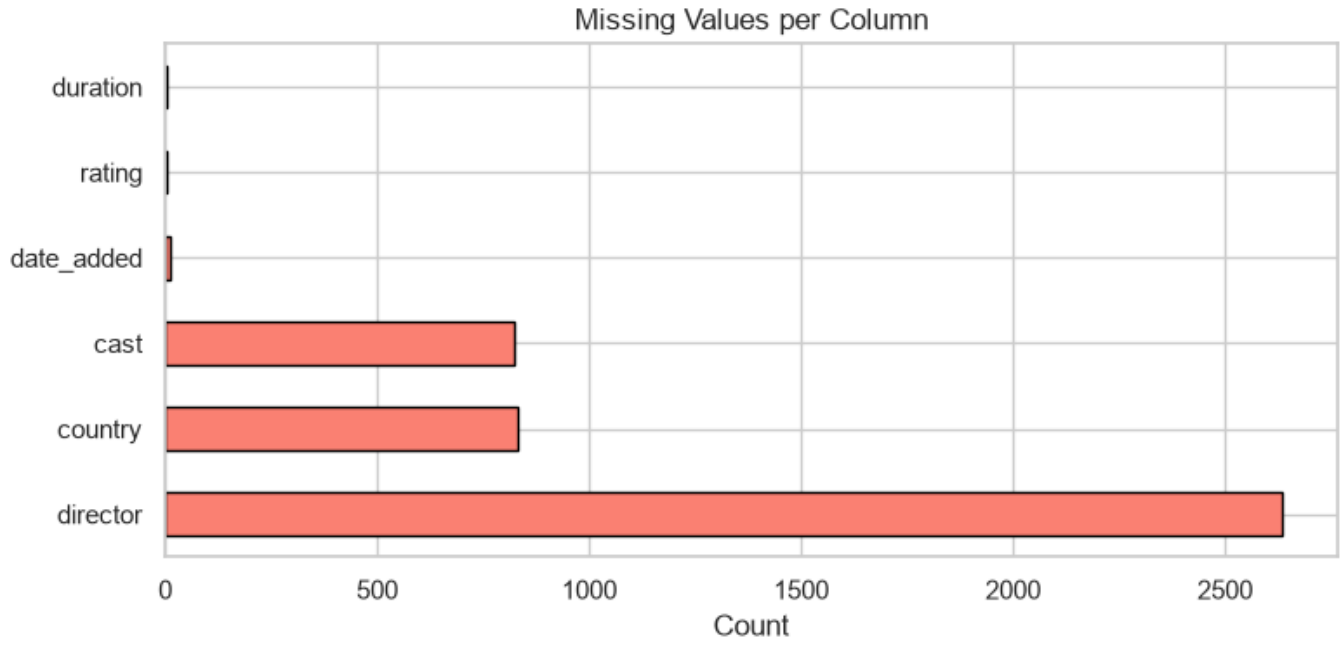
We check for missing values in each column, visualize them, and then apply appropriate strategies:
- director, cast, country: Fill with 'Unknown' (categorical, no reliable way to impute).
- date_added: Drop the few rows where this is missing since we need date parsing.
- rating, duration: Fill with mode or 'Unknown'.

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.DataFrame({'Missing Count': missing, 'Percentage (%)': missing_pct})
missing_df = missing_df[missing_df['Missing Count'] > 0].sort_values('Missing Count', ascending=False)
print(missing_df)
```

```
            Missing Count  Percentage (%)
director             2634          29.91
country               831           9.44
cast                  825           9.37
date_added             10           0.11
rating                  4           0.05
duration                3           0.03
```

```
fig, ax = plt.subplots(figsize=(8, 4))
missing_df['Missing Count'].plot(kind='barh', color='salmon', edgecolor='black', ax=ax)
ax.set_title('Missing Values per Column')
ax.set_xlabel('Count')
plt.tight_layout()
```

```
plt.show()
```




```
# Fill missing categorical values
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
df['rating'] = df['rating'].fillna(df['rating'].mode()[0])
df['duration'] = df['duration'].fillna(df['duration'].mode()[0])

# Drop rows where date_added is missing (very few)
print(f"Rows before dropping missing date_added: {len(df)}")
df = df.dropna(subset=['date_added'])
print(f"Rows after: {len(df)}")

# Verify no missing values remain
print(f"\nRemaining missing values: {df.isnull().sum().sum()}")
```

```
Rows before dropping missing date_added: 8807
Rows after: 8797

Remaining missing values: 0
```

6. Duplicate Detection and Removal

We check for duplicate rows based on the title and type columns -- a title can appear only once as a Movie and once as a TV Show.

```
dup_count = df.duplicated(subset=['title', 'type']).sum()
print(f"Duplicate rows (by title + type): {dup_count}")

if dup_count > 0:
    df = df.drop_duplicates(subset=['title', 'type'], keep='first')
    print(f"Rows after removing duplicates: {len(df)}")
else:
    print("No duplicates found -- no removal needed.")
```

```
Duplicate rows (by title + type): 0
No duplicates found -- no removal needed.
```

7. Date Parsing and Data Preprocessing

We parse date_added into a proper datetime, extract useful components (year, month, day of week), and split duration into numeric columns for Movies (minutes) and TV Shows (seasons).

```
# Parse date_added
df['date_added'] = pd.to_datetime(df['date_added'].str.strip())
df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month
df['day_of_week'] = df['date_added'].dt.day_name()

# Parse duration
df['duration_int'] = df['duration'].str.extract(r'(\d+)').astype(float)

print("Preprocessing complete.")
print(f"\nNew columns: year_added, month_added, day_of_week, duration_int")
df[['title', 'date_added', 'year_added', 'month_added', 'day_of_week', 'duration', 'duration_int']].head(10)
```

```
Preprocessing complete.

New columns: year_added, month_added, day_of_week, duration_int


                           title date_added  year_added  month_added  \
0             Dick Johnson Is Dead 2021-09-25        2021            9
1                    Blood & Water 2021-09-24        2021            9
2                        Ganglands 2021-09-24        2021            9
3            Jailbirds New Orleans 2021-09-24        2021            9
4                     Kota Factory 2021-09-24        2021            9
5                    Midnight Mass 2024-09-24        2021            9
6  My Little Pony: A New Generation 2021-09-24        2021            9
7                          Sankofa 2021-09-24        2021            9
8    The Great British Baking Show 2021-09-24        2021            9
9                     The Starling 2021-09-24        2021            9

  day_of_week   duration  duration_int
0    Saturday     90 min          90.0
1      Friday  2 Seasons           2.0
2      Friday   1 Season           1.0
3      Friday   1 Season           1.0
4      Friday  2 Seasons           2.0
5      Friday   1 Season           1.0
6      Friday     91 min          91.0
7      Friday    125 min         125.0
8      Friday  9 Seasons           9.0
9      Friday    104 min         104.0
```

8. Movies versus TV Shows

Let's look at the overall content split between Movies and TV Shows on Netflix.

```
type_counts = df['type'].value_counts()
print(type_counts)
print(f"\nMovie percentage: {type_counts.get('Movie',0)/len(df)*100:.1f}%")
print(f"TV Show percentage: {type_counts.get('TV Show',0)/len(df)*100:.1f}%")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

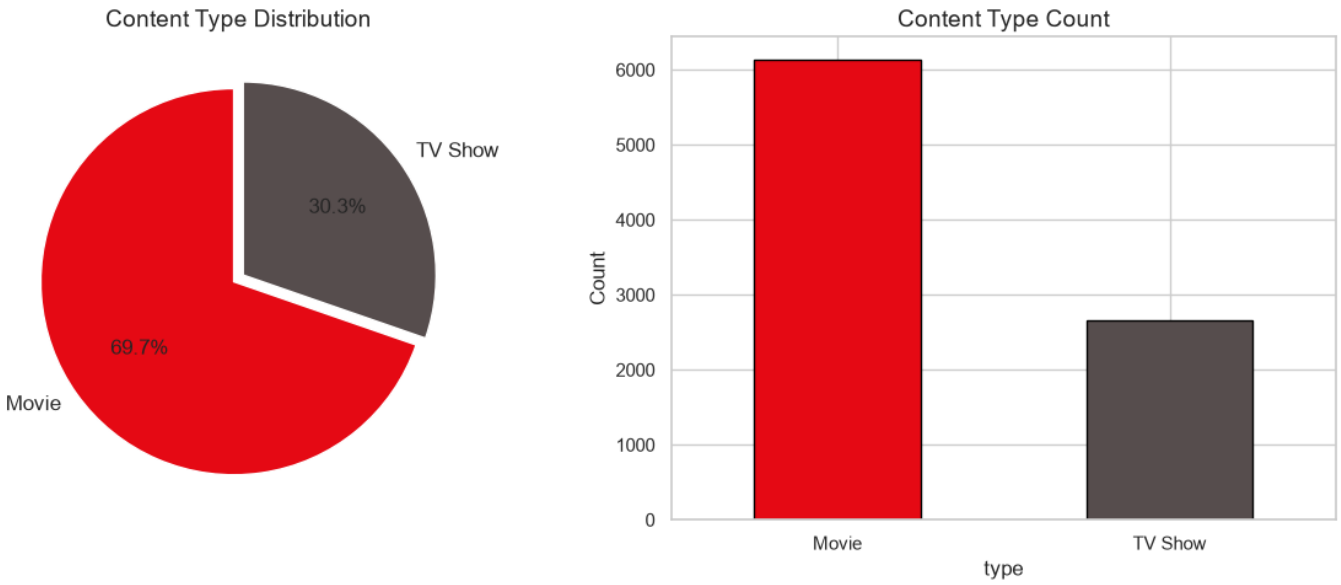
```
# Pie chart
colors = ['#E50914', '#564d4d']
axes[0].pie(type_counts, labels=type_counts.index, autopct='%1.1f%%',
            colors=colors, startangle=90, explode=[0.03, 0.03],
            textprops={'fontsize': 12})
axes[0].set_title('Content Type Distribution', fontsize=14)

# Bar chart
type_counts.plot(kind='bar', color=colors, edgecolor='black', ax=axes[1])
axes[1].set_title('Content Type Count', fontsize=14)
axes[1].set_ylabel('Count')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=0)

plt.tight_layout()
plt.show()
```

```
type
Movie      6131
TV Show    2666
Name: count, dtype: int64

Movie percentage: 69.7%
TV Show percentage: 30.3%
```



9. Genre Analysis

The listed_in column contains comma-separated genres. We split them to find the most common genres on Netflix.

```
# Split genres
genres = df['listed_in'].str.split(', ').explode()
genre_counts = genres.value_counts().head(15)

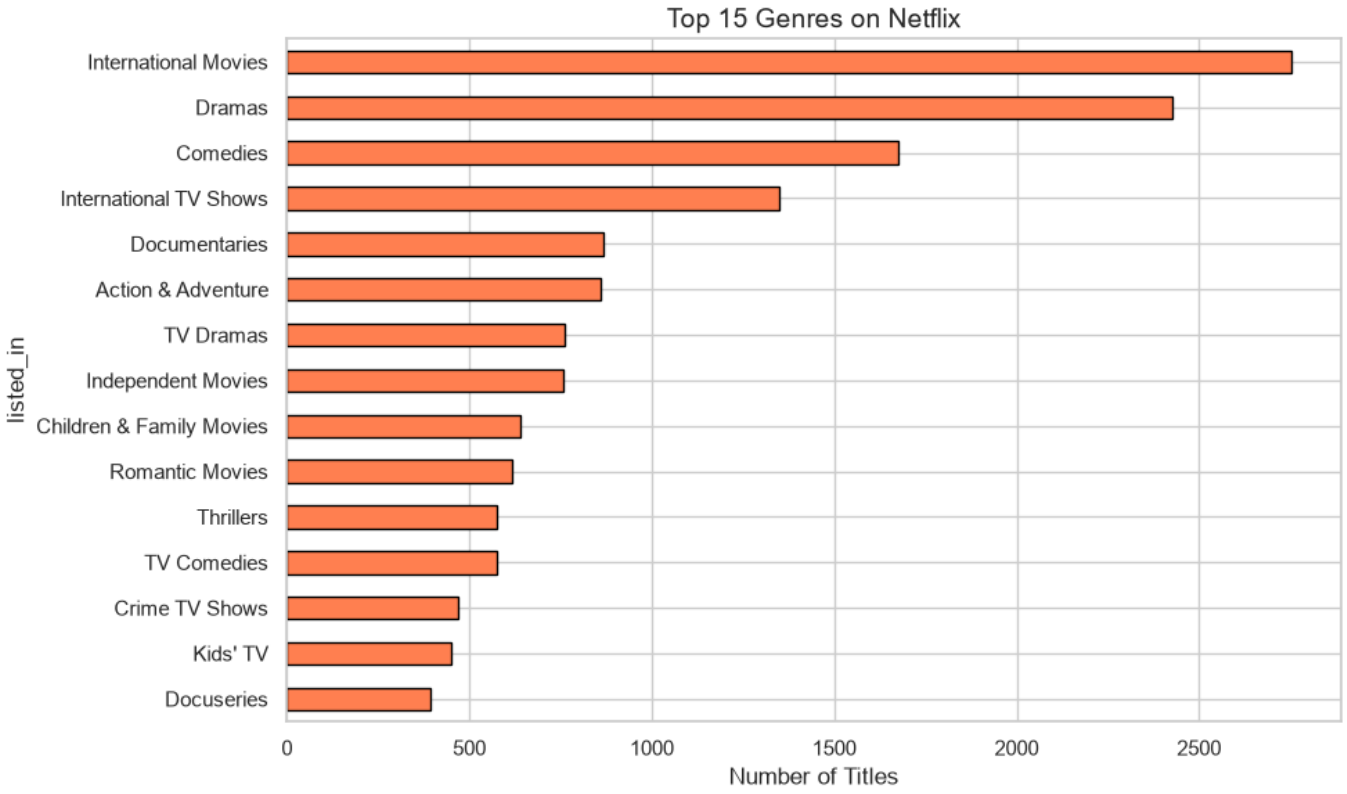
print("Top 15 Genres:\n")
print(genre_counts)
```

```
Top 15 Genres:

listed_in
International Movies        2752
Dramas                      2427
```

```
Comedies                   1674
International TV Shows     1350
Documentaries               869
Action & Adventure          859
TV Dramas                   762
Independent Movies          756
Children & Family Movies    641
Romantic Movies             616
Thrillers                   577
TV Comedies                 574
Crime TV Shows              469
Kids' TV                    449
Docuseries                  394
Name: count, dtype: int64
```

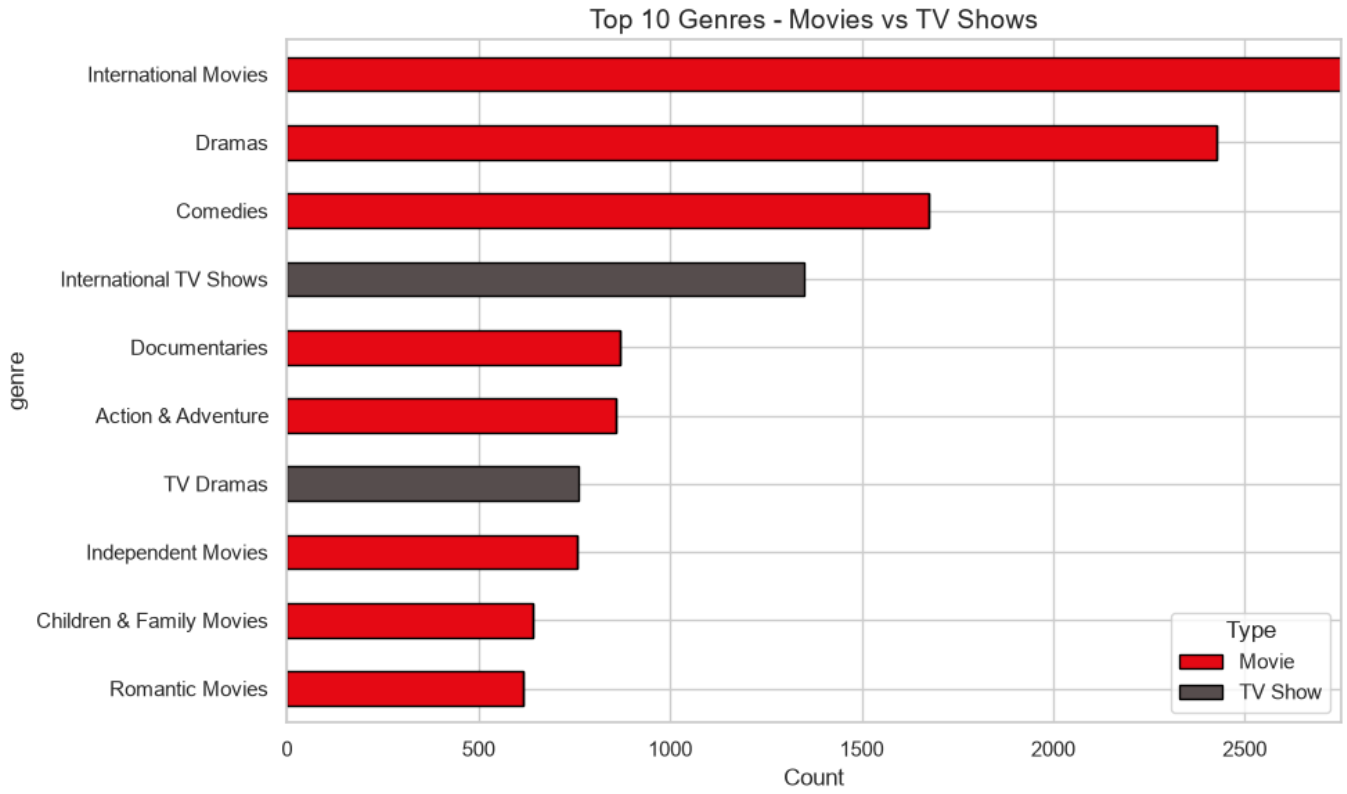
```
fig, ax = plt.subplots(figsize=(10, 6))
genre_counts.plot(kind='barh', color='coral', edgecolor='black', ax=ax)
ax.set_title('Top 15 Genres on Netflix', fontsize=14)
ax.set_xlabel('Number of Titles')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```




```
# Genre split by content type
genre_type = df[['listed_in', 'type']].copy()
genre_type = genre_type.assign(genre=genre_type['listed_in'].str.split(', ')).explode('genre')
top_genres = genre_type['genre'].value_counts().head(10).index.tolist()
genre_type_filtered = genre_type[genre_type['genre'].isin(top_genres)]

ct = genre_type_filtered.groupby(['genre', 'type']).size().unstack(fill_value=0)
ct = ct.loc[ct.index.isin(top_genres)]
# Reorder to match top genres order
ct = ct.reindex([g for g in top_genres if g in ct.index])
```

```
fig, ax = plt.subplots(figsize=(10, 6))
ct.plot(kind='barh', stacked=True, color=['#E50914', '#564d4d'], edgecolor='black', ax=ax)
ax.set_title('Top 10 Genres - Movies vs TV Shows', fontsize=14)
ax.set_xlabel('Count')
ax.legend(title='Type')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



10. Country Analysis

Some titles list multiple countries. We split them and examine the top content-producing countries.

```
countries = df[df['country'] != 'Unknown']['country'].str.split(', ').explode().str.strip()
country_counts = countries.value_counts().head(15)

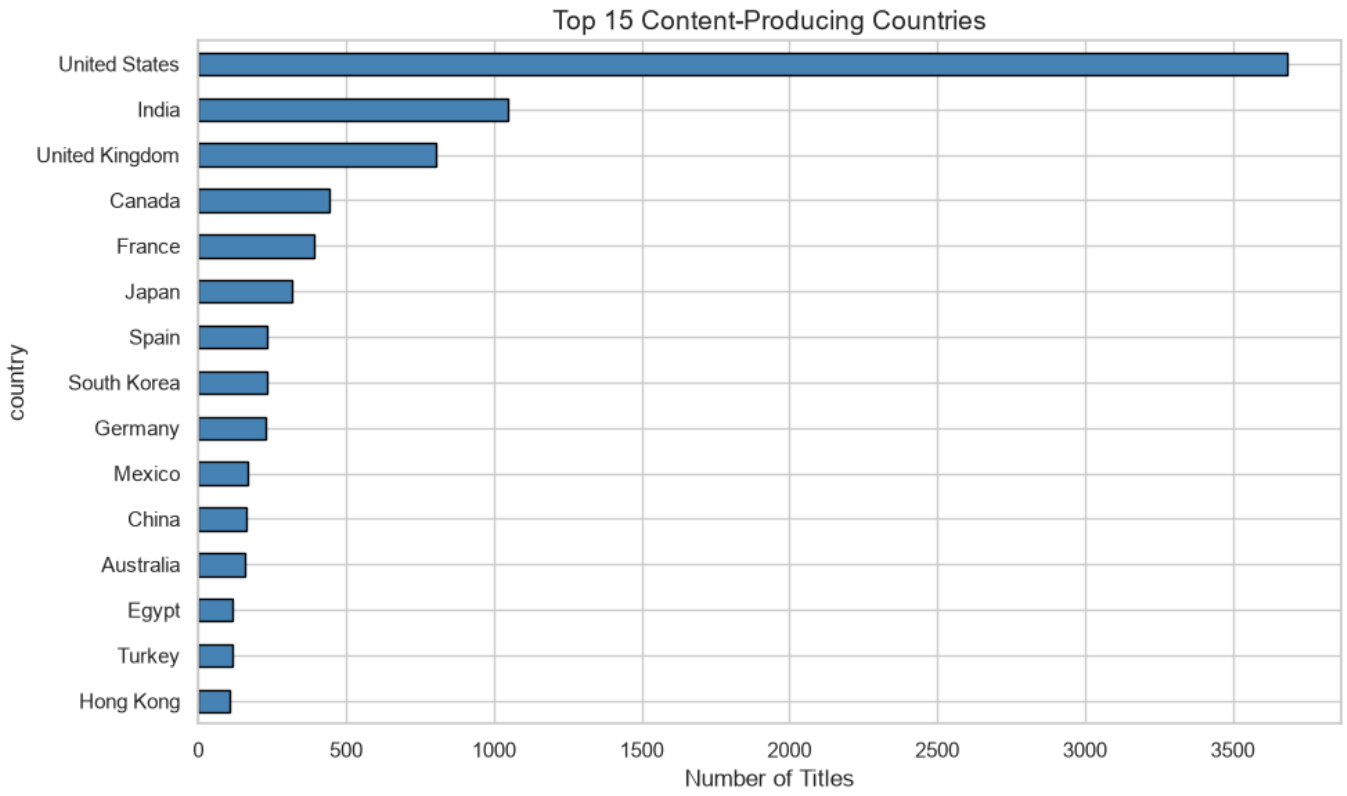
print("Top 15 Countries:\n")
print(country_counts)
```

```
Top 15 Countries:

country
United States     3683
India             1046
United Kingdom     803
Canada             445
France             393
Japan              317
Spain              232
South Korea        231
Germany            226
Mexico             169
China              162
```

```
Australia          159
Egypt              117
Turkey             113
Hong Kong          105
Name: count, dtype: int64
```

```
fig, ax = plt.subplots(figsize=(10, 6))
country_counts.plot(kind='barh', color='steelblue', edgecolor='black', ax=ax)
ax.set_title('Top 15 Content-Producing Countries', fontsize=14)
ax.set_xlabel('Number of Titles')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



11. Release-Year and Titles-Added-Year Trends

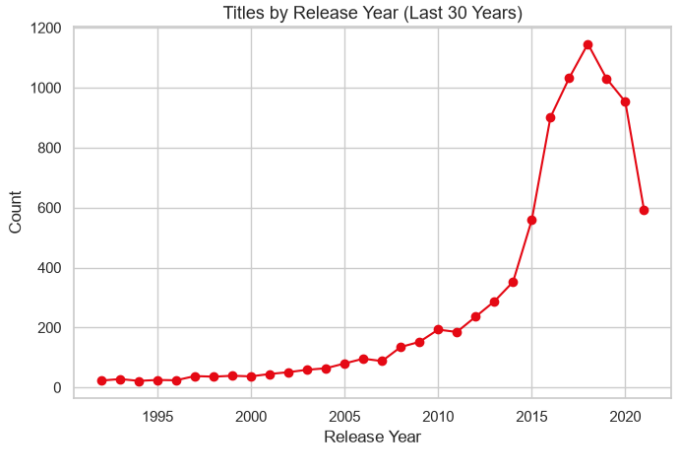
We look at when content was originally released and when it was added to the Netflix platform.

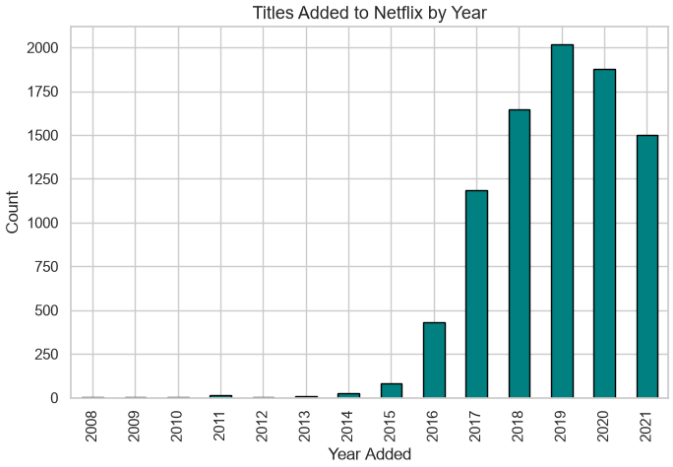
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Release year distribution
df['release_year'].value_counts().sort_index().tail(30).plot(
    kind='line', marker='o', color='#E50914', ax=axes[0])
axes[0].set_title('Titles by Release Year (Last 30 Years)', fontsize=13)
axes[0].set_xlabel('Release Year')
axes[0].set_ylabel('Count')

# Year added trend
year_added_counts = df['year_added'].value_counts().sort_index()
year_added_counts.plot(kind='bar', color='teal', edgecolor='black', ax=axes[1])
axes[1].set_title('Titles Added to Netflix by Year', fontsize=13)
axes[1].set_xlabel('Year Added')
axes[1].set_ylabel('Count')
```

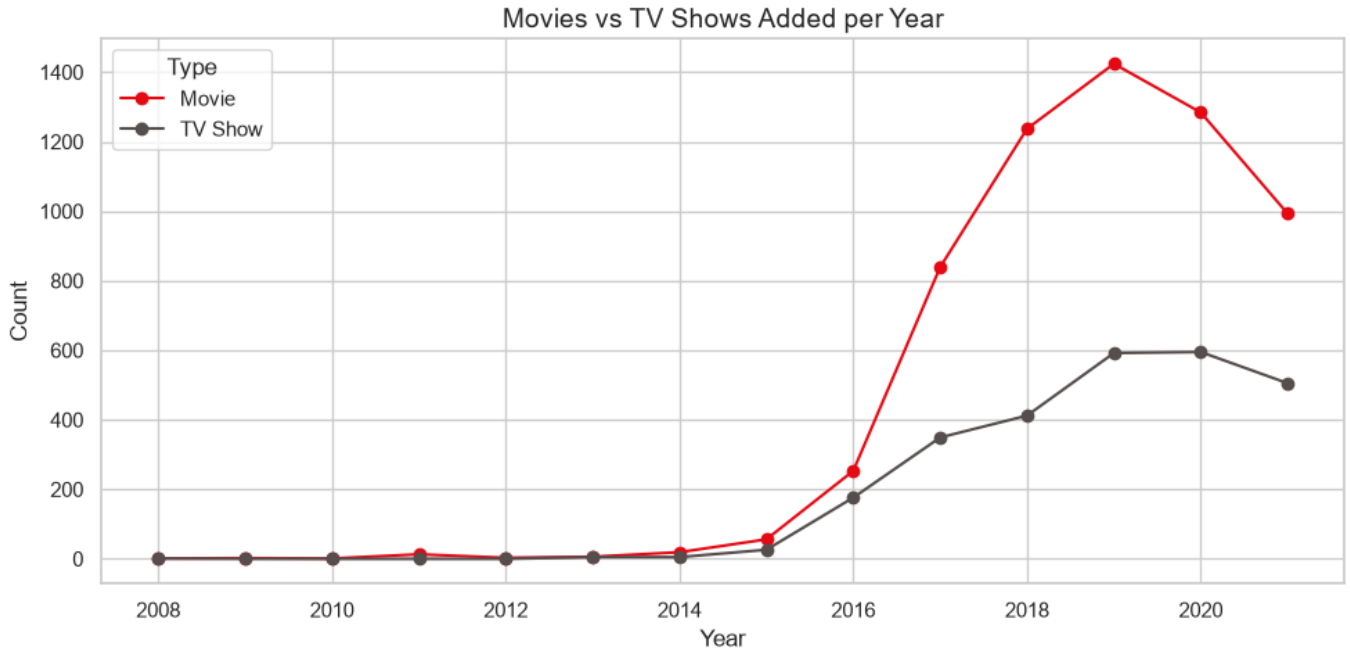
```
plt.tight_layout()
plt.show()
```






```
# Trend by type and year added
type_year = df.groupby(['year_added', 'type']).size().unstack(fill_value=0)

fig, ax = plt.subplots(figsize=(10, 5))
type_year.plot(kind='line', marker='o', color=['#E50914', '#564d4d'], ax=ax)
ax.set_title('Movies vs TV Shows Added per Year', fontsize=14)
ax.set_xlabel('Year')
ax.set_ylabel('Count')
ax.legend(title='Type')
plt.tight_layout()
plt.show()
```



12. Movie Duration and TV Show Season Analysis

For Movies we analyse the duration in minutes; for TV Shows we look at the number of seasons.

```
movies = df[df['type'] == 'Movie'].copy()
tv_shows = df[df['type'] == 'TV Show'].copy()

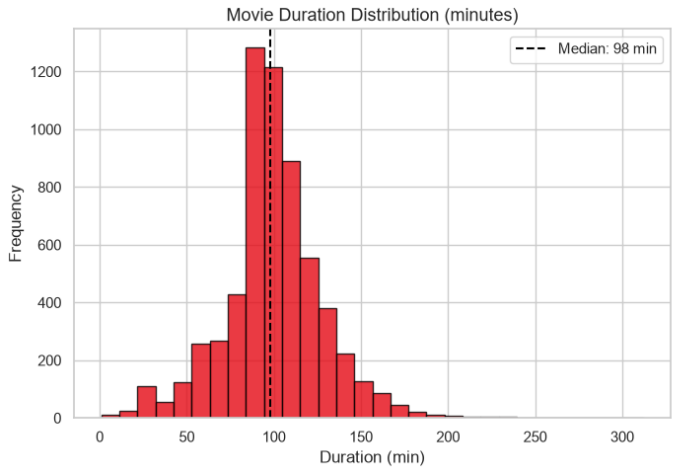
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

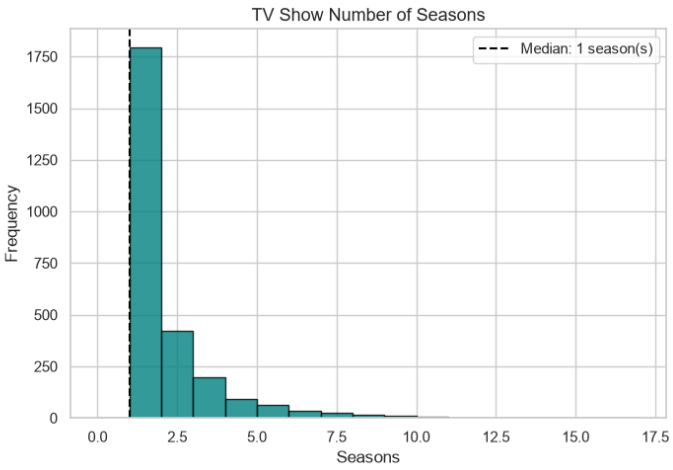
# Movie duration histogram
axes[0].hist(movies['duration_int'].dropna(), bins=30, color='#E50914',
             edgecolor='black', alpha=0.8)
axes[0].set_title('Movie Duration Distribution (minutes)', fontsize=13)
axes[0].set_xlabel('Duration (min)')
axes[0].set_ylabel('Frequency')
axes[0].axvline(movies['duration_int'].median(), color='black',
                linestyle='--', label=f"Median: {movies['duration_int'].median():.0f} min")
axes[0].legend()

# TV show seasons
axes[1].hist(tv_shows['duration_int'].dropna(), bins=range(0, 18),
             color='teal', edgecolor='black', alpha=0.8)
axes[1].set_title('TV Show Number of Seasons', fontsize=13)
axes[1].set_xlabel('Seasons')
axes[1].set_ylabel('Frequency')
axes[1].axvline(tv_shows['duration_int'].median(), color='black',
                linestyle='--', label=f"Median: {tv_shows['duration_int'].median():.0f} season(s)")
axes[1].legend()

plt.tight_layout()
plt.show()

print(f"Movie duration stats:\n{movies['duration_int'].describe().round(1)}\n")
print(f"TV Show season stats:\n{tv_shows['duration_int'].describe().round(1)}")
```






```
Movie duration stats:
count    6131.0
mean       99.5
std        28.4
min         1.0
25%        87.0
50%        98.0
75%       114.0
max       312.0
Name: duration_int, dtype: float64

TV Show season stats:
count    2666.0
mean        1.8
std         1.6
min         1.0
25%         1.0
50%         1.0
75%         2.0
max        17.0
Name: duration_int, dtype: float64
```

13. Content-Rating Analysis

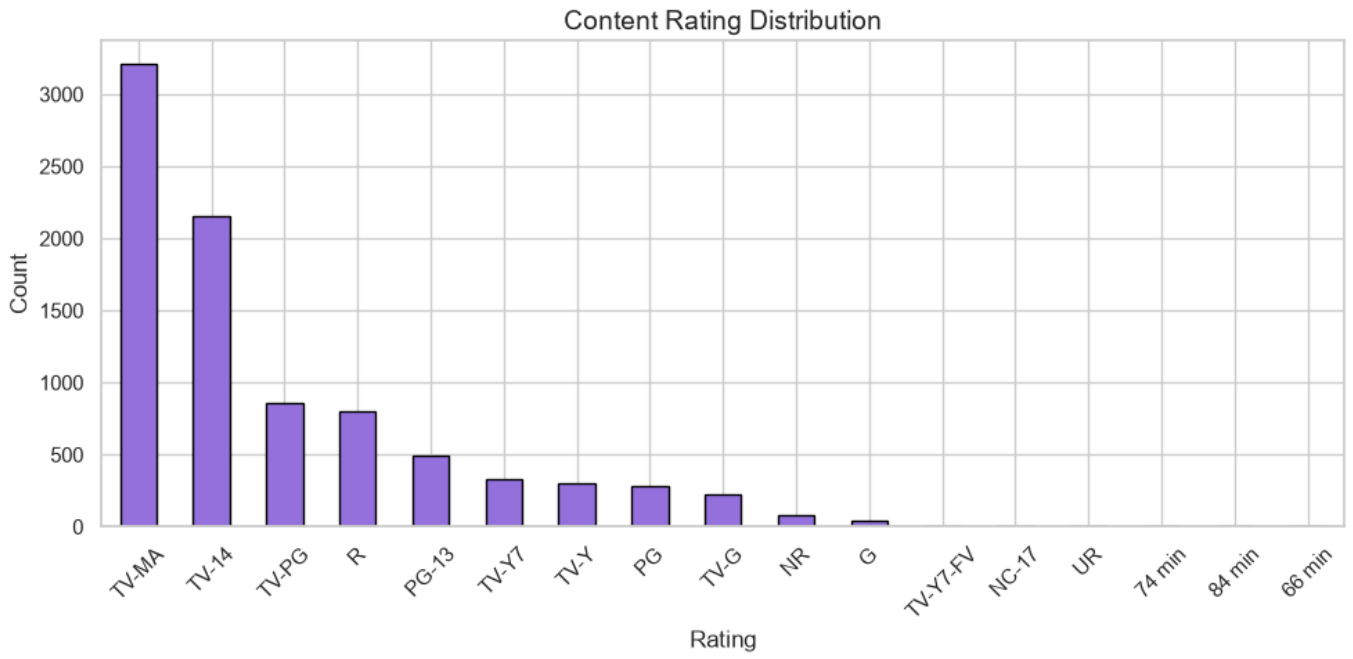
Netflix content carries maturity ratings like TV-MA, TV-14, PG-13, etc. Let's see how they are distributed.

```
rating_counts = df['rating'].value_counts()
print(rating_counts)
```

```
rating
TV-MA       3209
TV-14       2157
TV-PG        861
R            799
PG-13        490
TV-Y7        333
TV-Y         306
PG           287
TV-G         220
NR            79
G             41
TV-Y7-FV       6
NC-17          3
UR             3
```

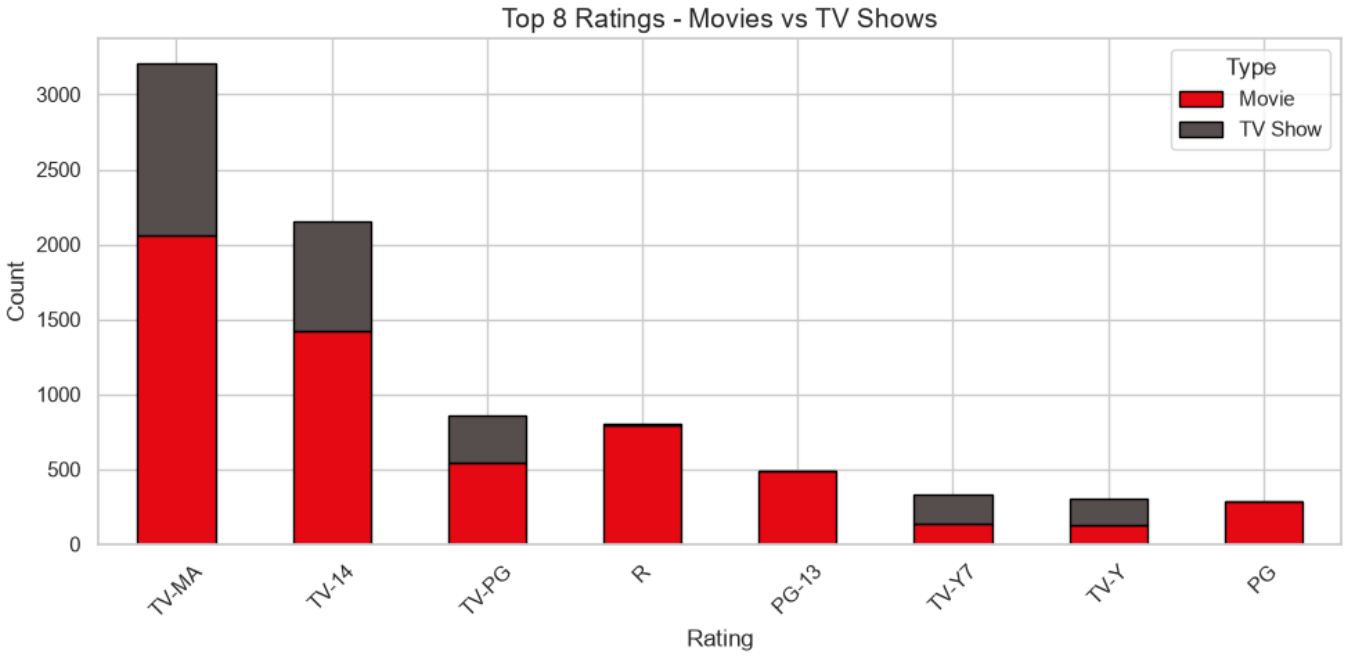
```
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64
```

```
fig, ax = plt.subplots(figsize=(10, 5))
rating_counts.plot(kind='bar', color='mediumpurple', edgecolor='black', ax=ax)
ax.set_title('Content Rating Distribution', fontsize=14)
ax.set_xlabel('Rating')
ax.set_ylabel('Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
# Rating by content type
top_ratings = rating_counts.head(8).index.tolist()
rating_type = df[df['rating'].isin(top_ratings)].groupby(['rating', 'type']).size().unstack(fill_value=0)
rating_type = rating_type.reindex([r for r in top_ratings if r in rating_type.index])

fig, ax = plt.subplots(figsize=(10, 5))
rating_type.plot(kind='bar', stacked=True, color=['#E50914', '#564d4d'],
                 edgecolor='black', ax=ax)
ax.set_title('Top 8 Ratings - Movies vs TV Shows', fontsize=14)
ax.set_xlabel('Rating')
ax.set_ylabel('Count')
ax.legend(title='Type')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



14. Outlier Detection Using IQR

We use the Interquartile Range (IQR) method to detect outliers in movie durations and release years.

IQR Method: Any value below Q1 - 1.5xIQR or above Q3 + 1.5xIQR is considered an outlier.

```
def detect_outliers_iqr(series, name):
    """Detect outliers using the IQR method and print summary."""
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = series[(series < lower) | (series > upper)]
    print(f"--- {name} ---")
    print(f"  Q1={Q1}, Q3={Q3}, IQR={IQR}")
    print(f"  Lower bound={lower}, Upper bound={upper}")
    print(f"  Outliers found: {len(outliers)} out of {len(series)} values")
    print(f"  Outlier percentage: {len(outliers)/len(series)*100:.2f}%\n")
    return lower, upper, outliers

# Movie duration outliers
l, u, movie_outliers = detect_outliers_iqr(movies['duration_int'].dropna(), 'Movie Duration (min)')

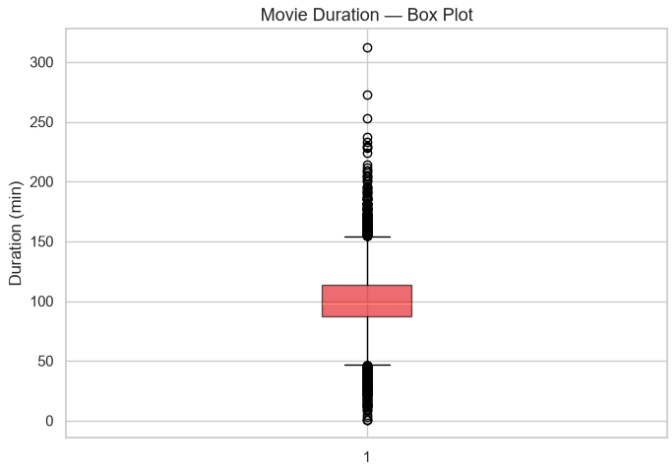
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].boxplot(movies['duration_int'].dropna(), vert=True, patch_artist=True,
                boxprops=dict(facecolor='#E50914', alpha=0.6))
axes[0].set_title('Movie Duration -- Box Plot', fontsize=13)
axes[0].set_ylabel('Duration (min)')

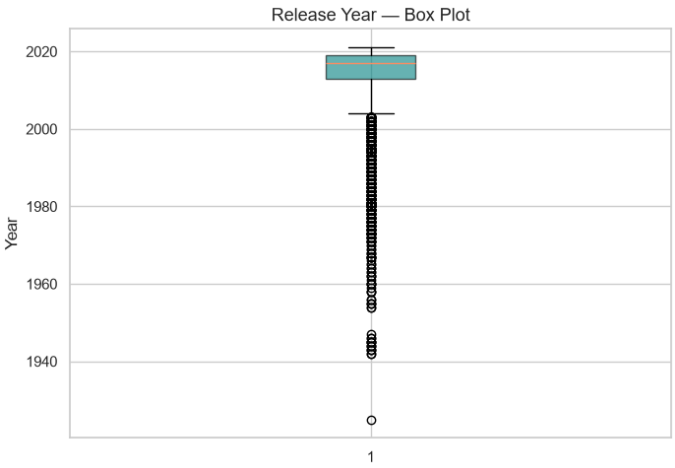
# Release year outliers
l2, u2, year_outliers = detect_outliers_iqr(df['release_year'], 'Release Year')
axes[1].boxplot(df['release_year'], vert=True, patch_artist=True,
                boxprops=dict(facecolor='teal', alpha=0.6))
axes[1].set_title('Release Year -- Box Plot', fontsize=13)
axes[1].set_ylabel('Year')

plt.tight_layout()
plt.show()
```

```
--- Movie Duration (min) ---
  Q1=87.0, Q3=114.0, IQR=27.0
  Lower bound=46.5, Upper bound=154.5
  Outliers found: 453 out of 6131 values
  Outlier percentage: 7.39%

--- Release Year ---
  Q1=2013.0, Q3=2019.0, IQR=6.0
  Lower bound=2004.0, Upper bound=2028.0
  Outliers found: 717 out of 8797 values
  Outlier percentage: 8.15%
```





15. Multivariate Analysis and Correlation

Since most columns are categorical, we compute correlations on the available numerical columns (release_year, duration_int, year_added, month_added) and supplement with cross-tabulation analysis.

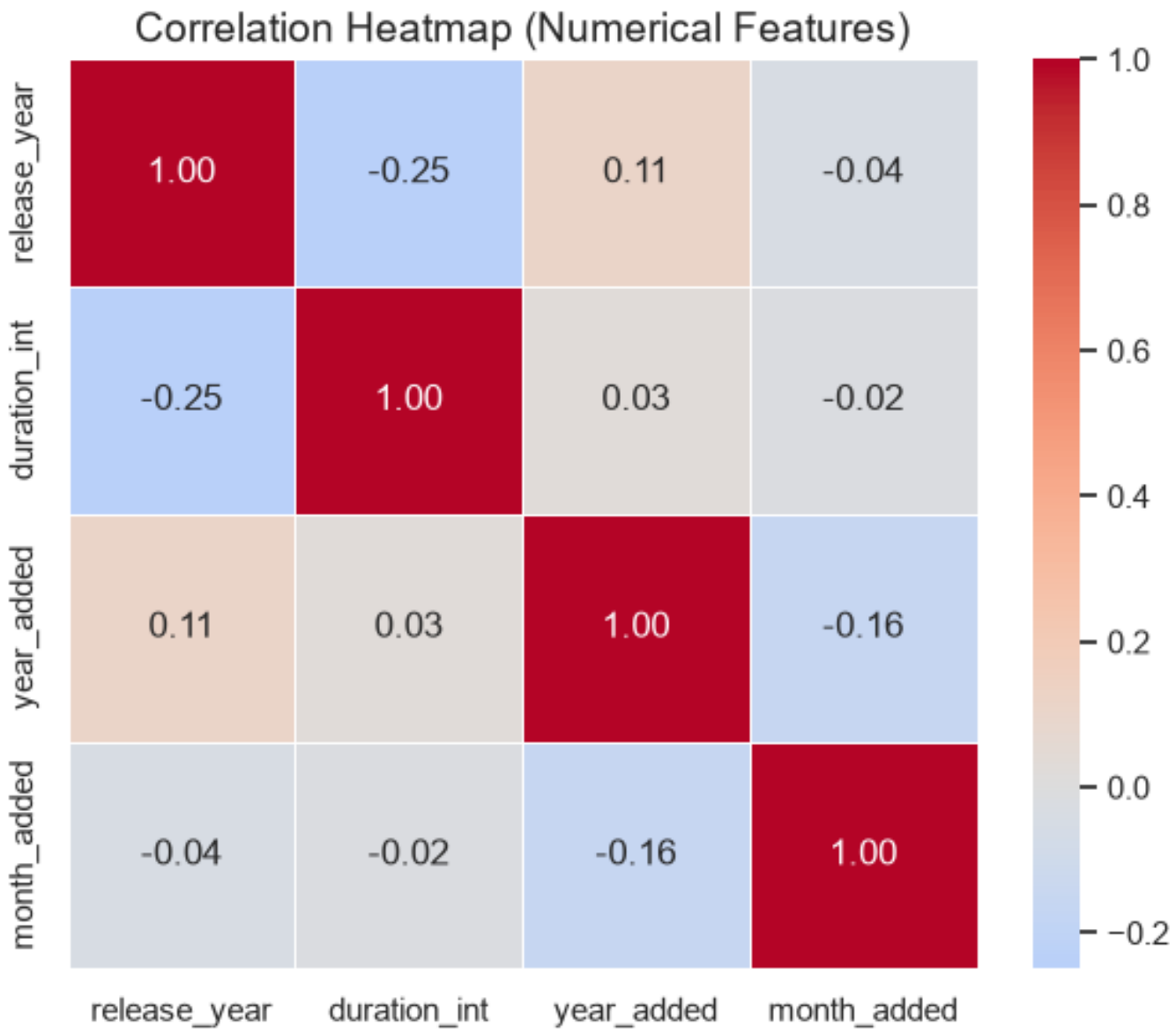
```
numeric_cols = df[['release_year', 'duration_int', 'year_added', 'month_added']].dropna()

corr = numeric_cols.corr()
print("Correlation matrix:\n")
print(corr.round(3))
```

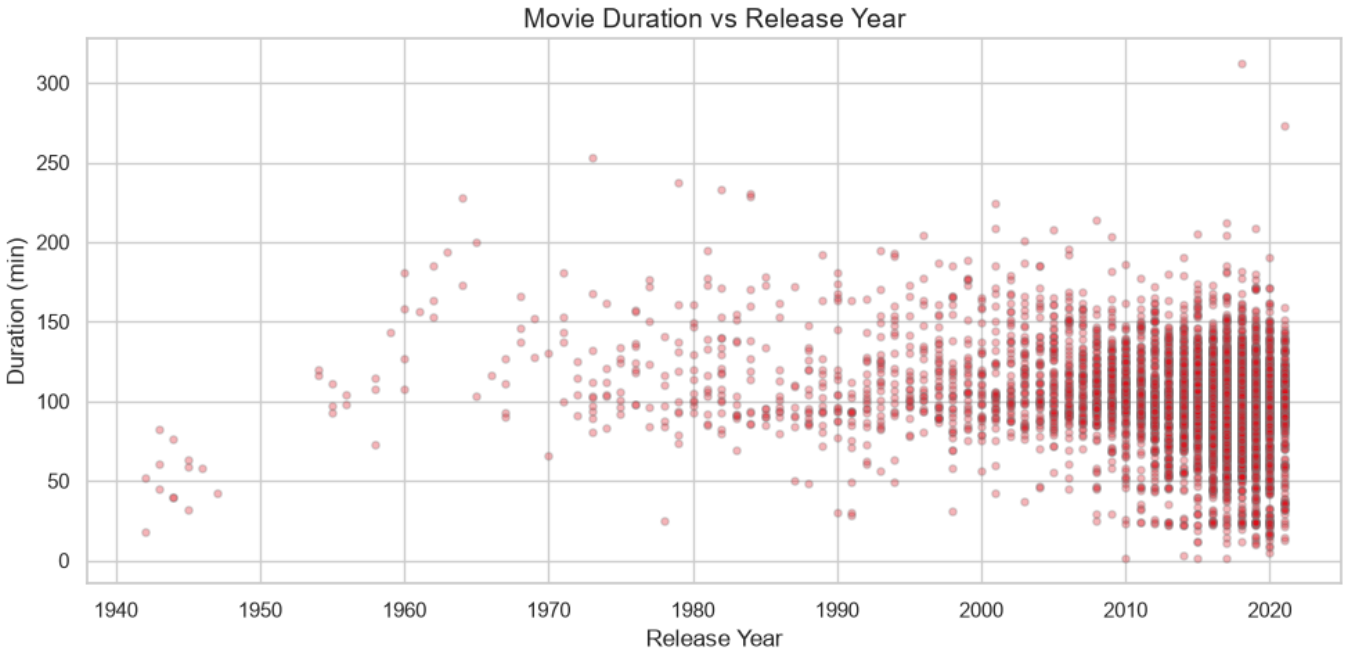
```
Correlation matrix:

              release_year  duration_int  year_added  month_added
release_year         1.000        -0.250       0.112       -0.039
duration_int        -0.250         1.000       0.030       -0.017
year_added           0.112         0.030       1.000       -0.160
month_added         -0.039        -0.017      -0.160        1.000
```

```
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=0.5, ax=ax)
ax.set_title('Correlation Heatmap (Numerical Features)', fontsize=14)
plt.tight_layout()
plt.show()
```

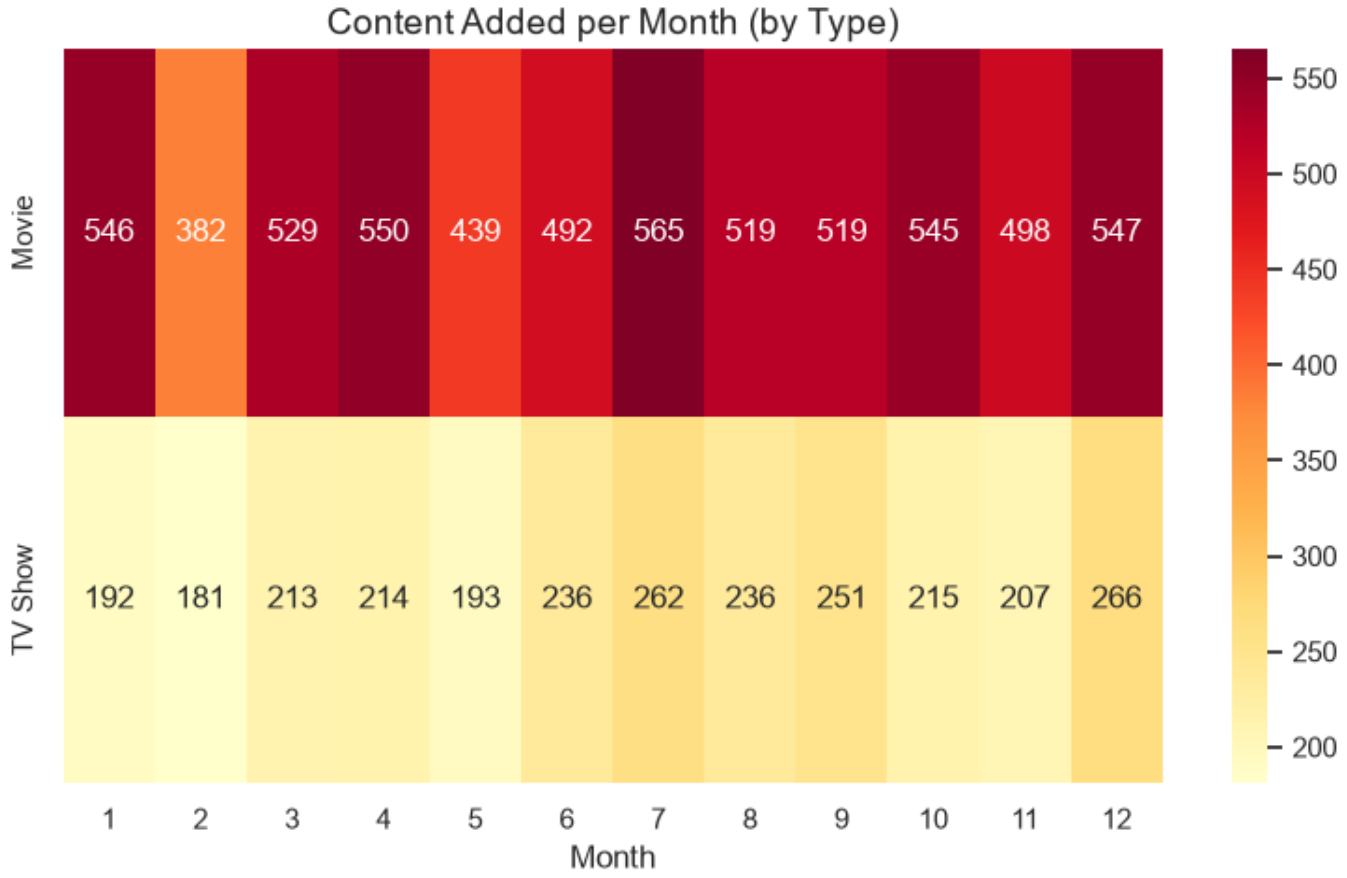



```
# Scatter: release year vs duration (Movies only)
fig, ax = plt.subplots(figsize=(10, 5))
ax.scatter(movies['release_year'], movies['duration_int'], alpha=0.3,
           color='#E50914', edgecolors='gray', s=15)
ax.set_title('Movie Duration vs Release Year', fontsize=14)
ax.set_xlabel('Release Year')
ax.set_ylabel('Duration (min)')
plt.tight_layout()
plt.show()
```

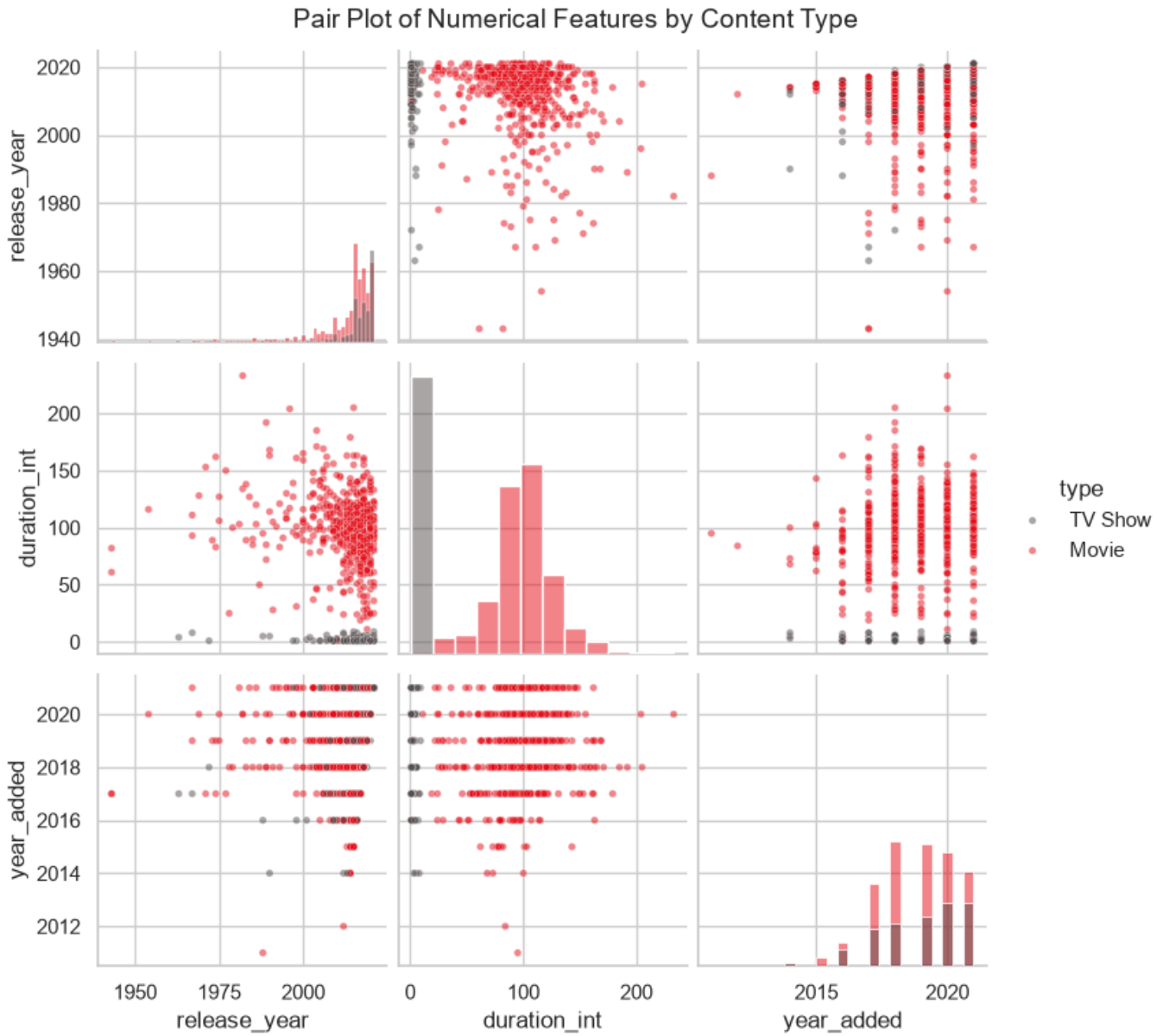



```
# Monthly content-addition heatmap by type
month_type = df.groupby(['month_added', 'type']).size().unstack(fill_value=0)

fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(month_type.T, annot=True, fmt='d', cmap='YlOrRd', ax=ax)
ax.set_title('Content Added per Month (by Type)', fontsize=14)
ax.set_xlabel('Month')
ax.set_ylabel('')
plt.tight_layout()
plt.show()
```




```
# Pair-plot of numerical features coloured by content type
# (sample to keep it fast)
sample = df[['release_year', 'duration_int', 'year_added', 'type']].dropna().sample(
    n=min(1000, len(df)), random_state=42)
g = sns.pairplot(sample, hue='type', palette={'Movie': '#E50914', 'TV Show': '#564d4d'},
                 diag_kind='hist', plot_kws={'alpha': 0.5, 's': 15})
g.figure.suptitle('Pair Plot of Numerical Features by Content Type', y=1.02, fontsize=14)
plt.show()
```



16. Findings, Limitations, and Conclusion

Key Findings

1. Content Split: Movies significantly outnumber TV Shows on Netflix (roughly 70-30 split).
2. Dominant Genres: International Movies, Dramas, and Comedies are the most common genres, reflecting Netflix's global strategy.
3. Geographic Concentration: The United States and India are the top content-producing countries by a large margin.
4. Temporal Trends: The bulk of content was added between 2016-2021, with a noticeable dip in 2020 (likely pandemic-related production delays) and a peak around 2019.
5. Movie Duration: Most movies are between 80-120 minutes, with a median around 100 minutes.
6. TV Shows: The vast majority of TV shows have only 1 season, indicating Netflix acquires many limited or single-season series.
7. Ratings: TV-MA is the most common rating, followed by TV-14, suggesting Netflix's catalogue skews toward adult-oriented content.
8. Outliers: Some very short films (< 20 min) and very long ones (> 180 min) exist as outliers.

Limitations

- The dataset is a snapshot (up to ~2021) and does not reflect current Netflix content.
- The director and cast fields have significant missing data, limiting cast/crew analysis.
- Genre labels are assigned by Netflix and may not follow standard film-industry categorizations.
- Multi-country productions are counted once per country when split, slightly inflating counts.

Conclusion

This EDA reveals that Netflix has built a content library that is predominantly movie-based and adult-oriented, with a strong global footprint, particularly in the US and India. The platform expanded its library aggressively between 2016-2020. The analysis also highlights data-quality challenges such as missing metadata. Future work could incorporate user-engagement metrics and more recent data to paint a fuller picture.

End of Notebook -- NetflixScope EDA